

Java Networking (Basics)

Java networking programming(or, Java Server Socket network programming)

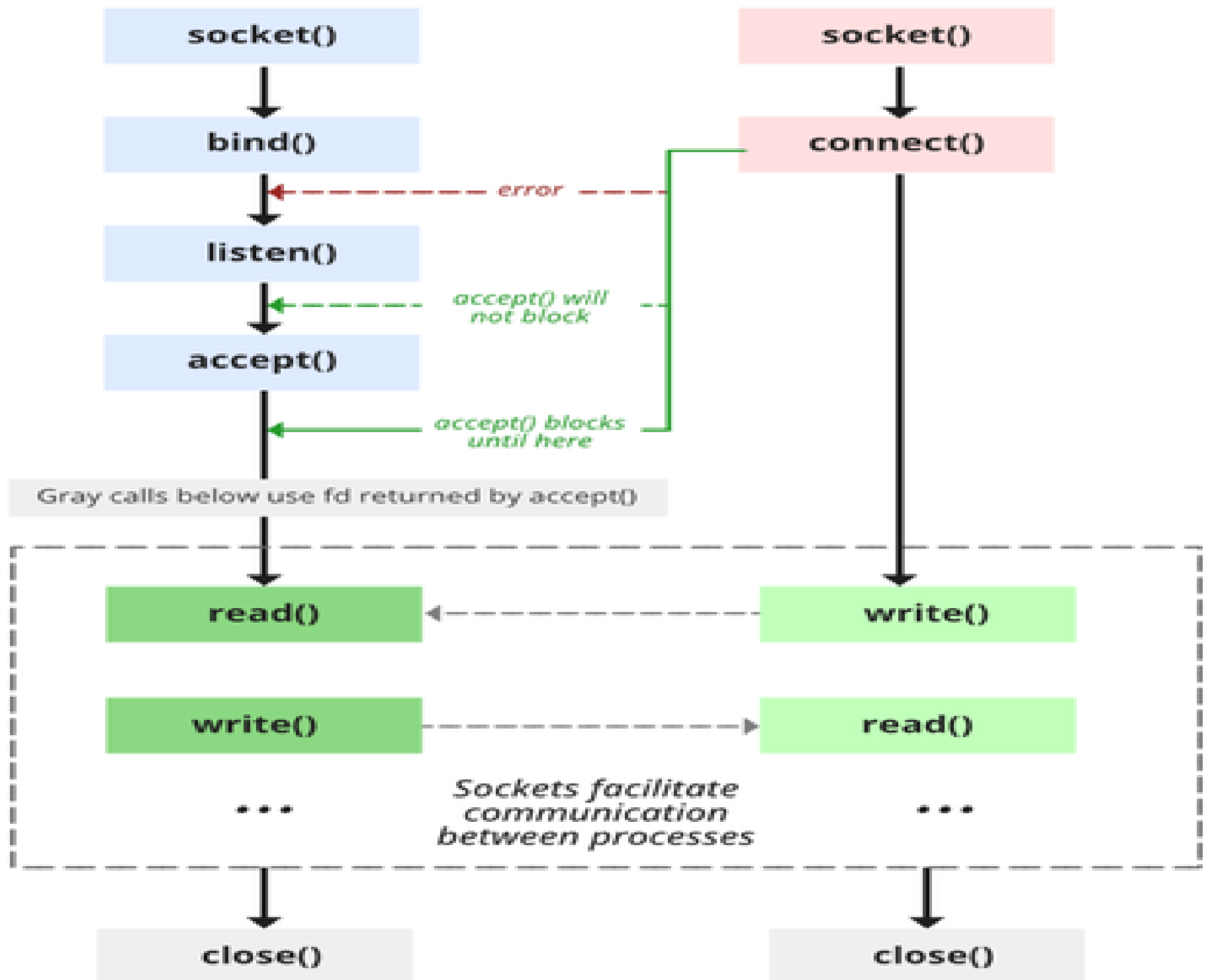
Socket programming in Java refers to the ability to create network communication between two devices or applications using sockets. Sockets are endpoints for sending or receiving data across a computer network. In Java, socket programming is primarily used for implementing client-server communication, where one program (the server) provides services and another program (the client) requests and uses those services.

Advantages of Java Networking

- Creating server-client applications
- Implementing networking protocols
- Implement socket programming
- Creating web services

Server Process

Client Process



State diagram for server and client model of Socket

Package Used in Networking

The `java.net` package of the J2SE APIs contains a collection of classes and interfaces that provide the low-level communication details, allowing you to write programs that focus on solving the problem given.

The `java.net` package provides support for the two common network protocols –

TCP – TCP stands for Transmission Control Protocol, which allows for reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.

UDP – UDP stands for User Datagram Protocol, a connection-less protocol that allows for packets of data to be transmitted between applications.

Socket Programming

- Sockets provide the communication mechanism between two computers using TCP. A client program creates a socket on its end of the communication and attempts to connect that socket to a server.
- When the connection is made, the server creates a socket object on its end of the communication. The client and the server can now communicate by writing to and reading from the socket.
- The `java.net.Socket` class represents a socket, and the `java.net.ServerSocket` class provides a mechanism for the server program to listen for clients and establish connections with them.

The following steps occur when establishing a TCP connection between two computers using sockets –



- The server instantiates a `ServerSocket` object, denoting which port number communication is to occur on.
- The server invokes the `accept()` method of the `ServerSocket` class. This method waits until a client connects to the server on the given port.
- After the server is waiting, a client instantiates a `Socket` object, specifying the server name and the port number to connect to.
- The constructor of the `Socket` class attempts to connect the client to the specified server and the port number. If communication is established, the client now has a `Socket` object capable of communicating with the server.
- On the server side, the `accept()` method returns a reference to a new socket on the server that is connected to the client's socket.



- After the connections are established, communication can occur using I/O streams. Each socket has both an `OutputStream` and an `InputStream`. The client's `OutputStream` is connected to the server's `InputStream`, and the client's `InputStream` is connected to the server's `OutputStream`.
- TCP is a two-way communication protocol, hence data can be sent across both streams at the same time. Following are the useful classes providing complete set of methods to implement sockets.

Advantages of Java Socket Programming

- ❑ **Platform Independence** – One of the biggest advantages of Java Sockets is that they are platform-independent. This means that the same Java code can be run on multiple operating systems and devices without the need for modification. This allows for easy deployment of network-based applications across different systems and ensures that the application can be run on different devices without the need for platform-specific code.
- ❑ **Easy to Use** – Java Sockets are also relatively easy to use, even for developers who are new to network programming. The Java API provides a simple, consistent interface for creating and managing sockets, which makes it easy to implement network-based applications without needing to understand the underlying network protocols.



- ❑ **Scalability** – Java Sockets are highly scalable, making them suitable for large-scale network-based applications. They can easily handle thousands of simultaneous connections and can be used to create distributed systems that can handle high levels of traffic.
- ❑ **Security** – Java Sockets provide built-in support for secure communication, including SSL and TLS encryption. This makes it easy to create secure network-based applications and ensures that sensitive data is protected while in transit.
- ❑ **Multithreading** – Java Sockets support multithreading, which means that multiple threads can be used to handle multiple connections simultaneously. This improves the performance of network-based applications and allows them to handle a large number of requests without becoming overloaded.



Disadvantages of Java Socket Programming

- ❑ **Complexity** – While Java Sockets are relatively easy to use, they can still be complex to implement. This complexity can make it difficult to debug and troubleshoot network-based applications, which can be time-consuming and frustrating.
- ❑ **Latency** – Java Sockets can introduce latency into network-based applications, particularly when dealing with large amounts of data. This can be a problem for applications that require real-time communication, such as online gaming or video conferencing.
- ❑ **Resource Intensive** – Java Sockets can be resource-intensive, particularly when dealing with large numbers of connections or large amounts of data. This can be a problem for systems with limited resources, such as mobile devices or embedded systems.



- ❑ Limited Protocol Support – Java Sockets support a limited number of network protocols, which can be a limitation for certain types of network-based applications. This can make it difficult to create applications that need to communicate with other systems using proprietary protocols.
- ❑ Potential for Security Vulnerabilities – Java Sockets, like any network-based application, are vulnerable to security threats, such as hacking and man-in-the-middle attacks. Careful attention must be paid to security when designing and implementing Java Socket-based systems to ensure that sensitive data is protected and potential vulnerabilities are identified and addressed.



Socket Programming Applications

- ❑ **Chat Applications** – Java Sockets are often used to create chat applications, such as instant messaging programs and online chat rooms. These types of applications typically use a client-server architecture, where clients connect to a central server to send and receive messages.
- ❑ **File Transfer Applications** – Java Sockets can also be used to create file transfer applications, such as peer-to-peer file sharing programs. These types of applications use a peer-to-peer architecture, where each device acts as both a client and a server. This allows for direct communication between devices, which can improve the speed and reliability of file transfers.

- ❑ **Remote Control Applications** – Java Sockets can also be used to create remote control applications, such as remote desktop software. These types of applications use a client-server architecture, where a client connects to a remote server to control the desktop of the server. This allows users to access and control their desktop from any device with an internet connection.
- ❑ **Multiplayer Games** – Java Sockets are also commonly used to create multiplayer games, such as online role-playing games and first-person shooters. These types of applications typically use a client-server architecture, where clients connect to a central server to play the game. The server acts as the intermediary between clients, handling communication and game logic.
- ❑ **IoT Applications** – Java Sockets can also be used in IoT (Internet of Things) applications, such as smart home systems. These types of applications use a client-server architecture, where IoT devices connect to a central server to send and receive data. This allows for remote monitoring and control of the devices, as well as data collection and analysis.



ServerSocket Class Constructors

The `java.net.ServerSocket` class is used by server applications to obtain a port and listen for client requests.

The `ServerSocket` class has four constructors –

1. `public ServerSocket(int port)` throws `IOException`

Attempts to create a server socket bound to the specified port. An exception occurs if the port is already bound by another application.

2. `public ServerSocket(int port, int backlog)` throws `IOException`

Similar to the previous constructor, the `backlog` parameter specifies how many incoming clients to store in a wait queue.

3. `public ServerSocket(int port, int backlog, InetAddress address)` throws `IOException`

Similar to the previous constructor, the `InetAddress` parameter specifies the local IP address to bind to. The `InetAddress` is used for servers that may have multiple IP addresses, allowing the server to specify which of its IP addresses to accept client requests on.



4. `public ServerSocket()` throws `IOException`

Creates an unbound server socket. When using this constructor, use the `bind()` method when you are ready to bind the server socket.

Note: If the `ServerSocket` constructor does not throw an exception, it means that your application has successfully bound to the specified port and is ready for client requests.

ServerSocket Class Methods

1. `public int getLocalPort()`

Returns the port that the server socket is listening on. This method is useful if you passed in 0 as the port number in a constructor and let the server find a port for you.

2. `public Socket accept() throws IOException`

Waits for an incoming client. This method blocks until either a client connects to the server on the specified port or the socket times out, assuming that the time-out value has been set using the `setSoTimeout()` method. Otherwise, this method blocks indefinitely.

3. `public void setSoTimeout(int timeout)`

Sets the time-out value for how long the server socket waits for a client during the `accept()`.



4. `public void bind(SocketAddress host, int backlog)`

Binds the socket to the specified server and port in the `SocketAddress` object. Use this method if you have instantiated the `ServerSocket` using the no-argument constructor.

Note : When the `ServerSocket` invokes `accept()`, the method does not return until a client connects. After a client does connect, the `ServerSocket` creates a new `Socket` on an unspecified port and returns a reference to this new `Socket`. A TCP connection now exists between the client and the server, and communication can begin.

Socket Class Constructors

The `java.net.Socket` class represents the socket that both the client and the server use to communicate with each other. The client obtains a `Socket` object by instantiating one, whereas the server obtains a `Socket` object from the return value of the `accept()` method.

1. `public Socket(String host, int port)` throws `UnknownHostException`, `IOException`.

This method attempts to connect to the specified server at the specified port. If this constructor does not throw an exception, the connection is successful and the client is connected to the server.

2. `public Socket(InetAddress host, int port)` throws `IOException`

This method is identical to the previous constructor, except that the host is denoted by an `InetAddress` object.



3. `public Socket(String host, int port, InetAddress localAddress, int localPort) throws IOException.`

Connects to the specified host and port, creating a socket on the local host at the specified address and port.

Socket Class Methods

1. `public void connect(SocketAddress host, int timeout)`
throws `IOException`

This method connects the socket to the specified host. This method is needed only when you instantiate the `Socket` using the no-argument constructor.

2. `public InetAddress getInetAddress()`

This method returns the address of the other computer that this socket is connected to.

3. `public int getPort()`

Returns the port the socket is bound to on the remote machine.

4. `public int getLocalPort()`

Returns the port the socket is bound to on the local machine.



5. **public SocketAddress getRemoteSocketAddress()**

Returns the address of the remote socket.

6. **public InputStream getInputStream() throws IOException**

Returns the input stream of the socket. The input stream is connected to the output stream of the remote socket.

7. **public OutputStream getOutputStream() throws IOException**

Returns the output stream of the socket. The output stream is connected to the input stream of the remote socket.

8. **public void close() throws IOException**

Closes the socket, which makes this Socket object no longer capable of connecting again to any server.

```

public class ChatServer
{
    public static void main(String args[]) throws Exception
    {
        ServerSocket ss=new ServerSocket(2020);
        Socket sk=ss.accept();
        BufferedReader cin=new BufferedReader(new
                                           InputStreamReader(sk.getInputStream()));
        PrintStream cout=new PrintStream(sk.getOutputStream());
        BufferedReader stdin=new BufferedReader(new
                                           InputStreamReader(System.in));

        String s;
        while ( true )
        {
            s=cin.readLine();
            if (s.equalsIgnoreCase("END"))
            {
                cout.println("BYE");
                break;
            }
            System.out.print("Client : "+s+"\n");
            System.out.print("Server : ");
            s=stdin.readLine();
            cout.println(s);
        }
        ss.close();
        sk.close();
        cin.close();
        cout.close();
        stdin.close();
    }
}

```

```
import java.net.*;
import java.io.*;
public class ChatClient
{
    public static void main(String args[]) throws Exception
    {
        Socket sk=new Socket("127.0.0.1",2020);
        BufferedReader sin=new BufferedReader(new
            InputStreamReader(sk.getInputStream()));

        PrintStream sout=new PrintStream(sk.getOutputStream());
        BufferedReader stdin=new BufferedReader(new
            InputStreamReader(System.in));

        String s;
        while ( true )
        {
            System.out.print("Client : ");
            s=stdin.readLine();
            sout.println(s);
            s=sin.readLine();
            System.out.print("Server : "+s+"\n");

            if(s.equalsIgnoreCase("BYE") )
            break;
        }
        sk.close();
        sin.close();
        sout.close();
        stdin.close();
    }
}
```

InetAddress Class Methods

This class represents an Internet Protocol (IP) address. Here are following useful methods which we would need while doing socket programming -

1. `static InetAddress getByAddress(byte[] addr)`

Returns an InetAddress object given the raw IP address.

2. `static InetAddress getByAddress(String host, byte[] addr)`

Creates an InetAddress based on the provided host name and IP address.

3. `static InetAddress getByName(String host)`

Determines the IP address of a host, given the host's name.



4. **String getAddress()**

Returns the IP address string in textual presentation.

5. **String getHostName()**

Gets the host name for this IP address.

6. **static InetAddress getLocalHost()**

Returns the local host.

7. **String toString()**

Converts this IP address to a String.

```
import java.net.InetAddress;  
import java.net.UnknownHostException;
```

```
public class InetAddressExample {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Get local host information
```

```
            InetAddress localhost = InetAddress.getLocalHost();
```

```
            System.out.println("Local Hostname: " + localhost.getHostName());
```

```
            System.out.println("Local IP Address: " + localhost.getHostAddress());
```

```
            // Get information for a specific hostname
```

```
            InetAddress microsoftAddress = InetAddress.getByName("www.microsoft.com");
```

```
            System.out.println("microsoft Hostname: " + microsoftAddress.getHostName());
```

```
            System.out.println("microsoft IP Address: " + microsoftAddress.getHostAddress());
```

```
            // Check if an address is reachable
```

```
            if (microsoftAddress.isReachable(5000)) { // 5-second timeout
```

```
                System.out.println("microsoft is reachable.");
```

```
            } else {
```

```
                System.out.println("microsoft is not reachable.");
```

```
            }
```

```
        } catch (UnknownHostException e) {
```

```
            System.err.println("Could not find host: " + e.getMessage());
```

```
        } catch (java.io.IOException e) {
```

```
            System.err.println("Network error: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Uniform Resource Identifier

A Uniform Resource Identifier (URI) is a unique string of characters that identifies a logical or physical resource on the internet or another network, such as a web page, an image, or a file.

URIs include both Uniform Resource Locators (URLs), which identify a resource by its location, and Uniform Resource Names (URNs), which identify a resource by its name.

URIs are the underlying mechanism for locating and interacting with online resources.



Key Aspects of a URI:

- **Identification:** Its primary purpose is to identify a specific resource on internet.
- **Uniqueness:** Each URI is a unique string of characters, ensuring that one resource is distinct from another.
- **Generality:** The term URI is a broad category that encompasses all forms of resource identifiers.
- **Interaction:** URIs enable internet protocols to facilitate interactions between and among various resources.

Java - URL Class

What is a URL?

The URL class in Java, found within the `java.net` package, represents a Uniform Resource Locator, which is essentially an address for a resource on the World Wide Web. It provides a way to interact with and extract information from these web addresses.

A URL can be broken down into parts, as follows –

Syntax:

`protocol://host:port/path?query#ref`

Example:

<https://www.oracle.com/index.htm?language=en#j2se>

Components of a URL:

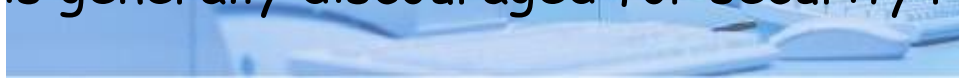
1. Scheme (Protocol):

- ❑ Indicates the protocol used to access the resource. Common examples include http, https, ftp, mailto, and file.
- ❑ It is followed by `://` (except for protocols like mailto, which use only a colon).

2. Authority (Optional, but common):

➤ User Information (Optional): user:password@

- ❑ Allows for embedding username and password for authentication, though this is generally discouraged for security reasons.



➤ Host:

- ❑ The domain name (e.g., www.example.com) or IP address (e.g., 192.168.1.1) of the server hosting the resource.

➤ Port (Optional): :port

- ❑ Specifies the network port number on the server. If omitted, default ports are used (e.g., 80 for HTTP, 443 for HTTPS).

3. Path (Optional): /path

- ❑ Indicates the specific location of the resource within the host's file system or server structure.
- ❑ Starts with a single forward slash /.

4. Query (Optional): ?query

- ❑ Provides additional parameters or data to the server, often used in search queries or dynamic content generation.
- ❑ Starts with a question mark ? and consists of name=value pairs separated by ampersands &.

5. Fragment (Optional): #fragment

- ❑ Points to a specific section or anchor within the resource, particularly in HTML documents.
- ❑ Starts with a hash symbol #. The browser navigates to the specified element ID within the page after loading.



URL - Constructor

1. `public URL(String protocol, String host, int port, String file)` throws `MalformedURLException`

Creates a URL by putting together the given parts.

2. `public URL(String protocol, String host, int port, String file, URLStreamHandler handler)` throws `MalformedURLException`

Creates a URL by putting together the given parts with the specified handler within a specified context.



URL Class Methods

The URL class contains many methods for accessing the various parts of the URL being represented.

Some of the methods in the URL class include the following -

1. **public equals(Object obj)**

This method compares this URL for equality with another object.

2. **public String getAuthority()**

This method returns the authority of the URL.

3. **public int getDefaultPort()**

This method returns the default port for the protocol of the URL.

4. **public String getFile()**

This method returns the filename of the URL.

5. **public String getHost()**

This method returns the host of the URL.

6. **public String getPath()**

This method returns the path of the URL.

7. **public int getPort()**

This method returns the port of the URL.

8. **public String getProtocol()**

This method returns the protocol of the URL.

9. **public String getQuery()**

This method returns the query part of the URL.

10. **public String getRef()**

This method returns the reference part of the URL.

11. **public URLConnection openConnection()**

This method returns a `URLConnection` instance that represents a connection to the remote **object** referred to by the URL.

12. **public String toString()**

This method constructs and return a string representation of this URL.

13. **public String toURI()**

This method returns a URI equivalent to this URL.

```
import java.net.*;
public class URLLDemo {

    public static void main(String [] args) {
        try {
            URL url = new
URL("https://www.oracle.com/index.htm?language=en#j2se");

            System.out.println("URL is " + url.toString());
            System.out.println("protocol is " + url.getProtocol());
            System.out.println("authority is " + url.getAuthority());
            System.out.println("file name is " + url.getFile());
            System.out.println("host is " + url.getHost());
            System.out.println("path is " + url.getPath());
            System.out.println("port is " + url.getPort());
            System.out.println("default port is " + url.getDefaultPort());
            System.out.println("query is " + url.getQuery());
            System.out.println("ref is " + url.getRef());
        }
        catch (URISyntaxException e)
            {
                e.printStackTrace();
            }
        catch (MalformedURLException e) {
            e.printStackTrace();
        }
    }
}
```

What is a URN?

- A URN (Uniform Resource Name) is another type of URI that names a resource instead of locating it.
- The focus of a URN is to provide a globally unique identifier for a resource, irrespective of where it is located.
- It is intended to remain globally unique and persistent even if the resource ceases to exist or becomes unavailable.

Example:

urn:isbn:0451450523 - This URN uniquely identifies a book using its ISBN number, irrespective of where or how the book can be accessed.

URLConnection Class

The `java.net.URLConnection` class in Java is an abstract class that serves as the superclass for all classes representing a communication link between an application and a URL. It allows for reading and writing data to the resource referenced by the URL.

Key aspects of `URLConnection`:

- ❑ **Establishing a Connection:** A `URLConnection` object is typically obtained by calling the `openConnection()` method on a `java.net.URL` object.
- ❑ **Reading and Writing Data:**
 - ❑ To read data from the connected resource, one can use `getInputStream()`, which returns an `InputStream`.
 - ❑ To write data to the resource, if supported by the protocol, one can use `getOutputStream()`, which returns an `OutputStream`.

- ❑ **Protocol-Specific Subclasses:** `URLConnection` is an abstract class, with concrete subclasses handling specific protocols. The most common subclass is `HttpURLConnection`, used for HTTP-based connections. Another example is `JarURLConnection` for connecting to JAR files.
- ❑ **Header Manipulation:** `URLConnection` provides methods to get and set various header fields for both client requests and server responses, enabling fine-grained control over communication.
- ❑ **Configuration Options:** It offers methods to configure connection parameters such as timeouts (`setConnectTimeout`, `setReadTimeout`), caching behavior (`setUseCaches`), and request properties (`setRequestProperty`).
- ❑ **Retrieving Resource Information:** Methods like `getContentLength()`, `getContentType()`, `getLastModified()`, and `getHeaderField()` allow retrieval of information about the resource from the server's response headers.

```
import java.net.URL;
import java.net.URLConnection;
import java.io.InputStream;
import java.io.BufferedReader;
import java.io.InputStreamReader;
```

```
public class URLConnectionExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://www.google.com");
            URLConnection connection = url.openConnection();

            // Get the input stream to read data
            InputStream inputStream = connection.getInputStream();
            BufferedReader reader = new BufferedReader(new
                                                        InputStreamReader(inputStream));

            String line;
            StringBuilder content = new StringBuilder();
            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }
            reader.close();
            inputStream.close();

            System.out.println("Content from URL:\n" + content.toString());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

URLConnection Class

`java.net.HttpURLConnection`, is an abstract class which represents the HTTP-specific URL connection. Instances of this class can be used both to read from and to write to the resource referenced by the URL.

This class extends following classes

- `java.lang.Object`
- `java.net.URLConnection`

For example -

If you connect to a URL whose protocol is HTTP, the `URL.openConnection()` method returns an `HttpURLConnection` object.

Steps to make a connection to a URL

Following are the steps to make a connection to the URL and start processing.

- Invoke `URL.openConnection()` method to get `URLConnection` object of an HTTP based URL.
- Update setup parameters and general request properties as required using connection object various setter methods.
- Create a connection to remote object using `connect()` method of connection object.
- Once remote object is available, access the content/headers of the remote object.

Example of Java HttpURLConnection Class

The following HttpURLConnection program connects to a URL given.

If the URL represents an HTTP resource, the connection is cast to HttpURLConnection, and the data in the resource is read one line at a time.

HttpURLConnection Class Declaration

This class extends following classes

`java.lang.Object`

`java.net.URLConnection`

`public abstract class HttpURLConnection extends URLConnection.`

Example of Java HttpURLConnection Class

The following HttpURLConnection program connects to a URL written in a program.

If the URL represents an HTTP resource, the connection is cast to HttpURLConnection, and the data in the resource is read one line at a time.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLConnection;
```

```
public class HttpURLConnectionDemo {
    public static void main(String [] args) {
        try {
            URL url = new URL("https://www.cimage.in");
            URLConnection urlConnection = url.openConnection();
            HttpURLConnection connection = null;
            if(urlConnection instanceof HttpURLConnection) {
                connection = (HttpURLConnection) urlConnection;
            }else {
                System.out.println("Please enter an HTTP URL.");
                return;
            }

            BufferedReader in = new BufferedReader(
                new InputStreamReader(connection.getInputStream()));
            String urlString = "";
            String current;

            while((current = in.readLine()) != null) {
                urlString += current;
            }
            System.out.println(urlString);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Datagram in JAVA

Datagram programming in Java utilizes the User Datagram Protocol (UDP) for connectionless communication, where data is sent in independent packets called datagrams. Unlike TCP, UDP does not guarantee delivery, order, or error-free transmission, making it suitable for applications where speed is prioritized over reliability (e.g., streaming, online gaming).

Java implements datagram programming using two primary classes from the `java.net` package:

- ❑ **DatagramPacket:** This class represents a datagram, encapsulating the data to be sent or received, along with the destination IP address and port number (for sending) or the source IP address and port number (for receiving).
- ❑ **DatagramSocket:** This class provides the mechanism to send and receive `DatagramPacket` objects. It acts as a communication endpoint on a specific port.

Key Steps for Datagram Programming (Client/Server Model)

1. Server-Side (Receiver):

- Create a `DatagramSocket`:

Bind it to a specific port to listen for incoming datagrams.

```
DatagramSocket serverSocket = new DatagramSocket(portNumber);
```

- Create a `DatagramPacket` for receiving:

Specify a byte array buffer and its length to store the incoming data.

```
byte[] receiveBuffer = new byte[1024];  
DatagramPacket receivePacket = new DatagramPacket(receiveBuffer,  
receiveBuffer.length);
```

- Receive the packet:

Use the `receive()` method of the `DatagramSocket`. This is a blocking call.

- `serverSocket.receive(receivePacket);`
`serverSocket.receive(receivePacket);`

- Extract data and sender information.

```
String receivedData = new String(receivePacket.getData(), 0,  
                                receivePacket.getLength());  
InetAddress senderAddress = receivePacket.getAddress();  
int senderPort = receivePacket.getPort();
```

- Close the socket.
serverSocket.close();

2. Client-Side (Sender):

Create a DatagramSocket: You can bind it to an arbitrary available port, or let the system assign one.

```
DatagramSocket clientSocket = new DatagramSocket();
```

- **Prepare data to send:** Convert it to a byte array.

```
String message = "Hello UDP Server!";  
byte[] sendData = message.getBytes();
```

- **Create a DatagramPacket for sending:**
Specify the data, its length, the destination IP address (InetAddress), and the destination port.

```
InetAddress serverAddress =  
InetAddress.getByName("localhost"); // or server IP  
int serverPort = 8876;
```



```
DatagramPacket sendPacket = new DatagramPacket(sendData,  
sendData.length, serverAddress, serverPort);
```

- **Send the packet.**
clientSocket.send(sendPacket);
- **Close the socket.**
clientSocket.close();

Server-side code

```
public class UDPServer
{
    public static void main(String[] args) throws IOException
    {
        DatagramSocket socket = new DatagramSocket(3000);
        byte[] buffer = new byte[1024];
        DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

        while (true)
        {
            socket.receive(packet);
            System.out.println("Received: " + new String(packet.getData()));
            socket.send(packet); // echo back
        }
    }
}
```

client-side code

```
public class UDPClient
{
    public static void main(String[] args) throws IOException
    {
        DatagramSocket socket = new DatagramSocket();

        String message = "Hello, Server!";

        byte[] buffer = message.getBytes();

        InetAddress address = InetAddress.getByName("localhost");

        DatagramPacket packet = new DatagramPacket(buffer, buffer.length,
address, 3000);

        socket.send(packet);

        socket.receive(packet);

        System.out.println("Received: " + new String(packet.getData()));

        socket.close();
    }
}
```

